WIKIPEDIA
25 years of the free encyclopedia

decimal64 floating-point format

In computing, **decimal64** is a decimal floating-point computer number format that occupies 8 bytes (64 bits) in computer memory. The format was formally introduced in the 2008 revision[1] of the IEEE 754 standard, also known as ISO/IEC/IEEE 60559:2011.[2]

Format

Decimal64 values are categorized as normal or subnormal (denormal) numbers and can be encoded in either binary integer decimal (BID) or densely packed decimal (DPD) formats. Normal values can possess 16-digit precision ranging from $\pm 1.000000000000000 \times 10^{-383}$ to $\pm 9.999999999999999 \times 10^{384}$. In addition to normal and subnormal numbers, the format also includes signed zeros, infinities, and NaNs.

The binary format of identical size accommodates a spectrum from denormal-min $\pm 5 \times 10^{-324}$, through normal-min with complete 53-bit precision $\pm 2.2250738585072014 \times 10^{-308}$, to maximum $\pm 1.7976931348623157 \times 10^{+308}$.

Because the significand for the IEEE 754, decimal formats are not normalized and most values with less than 16 significant digits have multiple possible representations; $1000000 \times 10^{-2} = 100000 \times 10^{-1} = 10000 \times 10^{0} = 1000 \times 10^{1}$ all have the value 10000. These sets of representations for the same value are called *cohorts*. The different members can be used to denote how many digits of the value are known precisely. Each signed zero has 768 possible representations (1536 for all zeros, in two different cohorts).

Encoding of decimal64 values

Sign	Combination	Significand continuation
1 bit	13 bits	50 bits

IEEE 754 allows two alternative encodings for decimal64 values. The standard does not specify how to signify which representation is used. For instance, in a situation where decimal64 values are communicated between systems:

- In the binary encoding, the 16-digit significand is represented as a binary coded positive integer, based on binary integer decimal (BID).
- In the decimal encoding, the 16-digit significand is represented as a decimal coded positive integer, based on densely packed decimal (DPD) with 5 groups of 3 digits each represented in *declets*, 10-bit sequences. The most significant bit is encoded separately. This is efficient because $2^{10} = 1024$, is only slightly more than needed to contain all integers from 0 to 999.

Both alternatives provide exactly the same set of representable numbers: 16 digits of significand and $3 \times 2^8 = 768$ possible decimal exponent values. All the possible decimal exponent values storable in a binary64 number are representable in decimal64, and most bits of the significand of a binary64 are stored keeping roughly the same number of decimal digits in the significand.

In both cases, the most significant 4 bits of the significand, which only have 10 possible values, are combined with two bits of the exponent (3 possible values) to use 30 of the 32 possible values of a 5-bit field. The remaining combinations encode infinities and NaNs. BID and DPD use different bits of the combination field.

For Infinity and NaN, all other bits of the encoding are not used. Thus, an array can be filled with a single byte value to set it to Infinities or NaNs.

Binary integer significand field

This format uses a binary significand from 0 to $10^{16} - 1 = 9\,999\,999\,999\,999\,999 = 2386F26FC0FFFF_{16} = 1000\ 1110000110\ 1111001001\ 1011111100\ 0000111111\ 1111111111_2$.The encoding, completely stored on 64 bits, can represent binary significands up to $10 \times 2^{50} - 1 = 11\,258\,999\,068\,426\,239 = 27FFFFFFFFFFFF_{16}$, but values larger than $10^{16} - 1$ are illegal and the standard requires implementations to treat them as 0 if encountered on input.

As described above, the encoding varies depending on whether the most significant 4 bits of the significand are in the range 0 to 7 (0000_2 to 0111_2), or higher (1000_2 or 1001_2).

If the two bits after the sign bit are "00", "01", or "10", then the exponent field consists of the 10 bits following the sign bit and the significand is the remaining 53 bits with an implicit leading 0 bit. This includes subnormal numbers where the leading significand digit is 0.

If the 2 bits after the sign bit are "11", then the 10-bit exponent field is shifted 2 bits to the right (after both the sign bit and the "11" bits thereafter) and the represented significand is in the remaining 51 bits. In this case there is an implicit (that is, not stored) leading 3-bit sequence "100" for the MSB bits of the true significand (in the remaining lower bits *ttt...ttt* of the significand, not all possible values are used).

BID Encoding

Combination Field														Exponent	Significand / Description
g12	g11	g10	g9	g8	g7	g6	g5	g4	g3	g2	g1	g0			
combination field not starting with '11', bits ab = 00, 01 or 10															
a	**b**	**c**	**d**	**m**	**m**	**m**	**m**	**m**	**m**	**e**	**f**	**g**		**abcdmmmmmm**	(0)**efgttttttttttttttttttttttttttttttttttttttttttttttttttttt** Finite number with small first digit of significand (0 .. 7).
combination field starting with '11', but not 1111, bits ab = 11, bits cd = 00, 01 or 10															
1	1	**c**	**d**	**m**	**m**	**m**	**m**	**m**	**m**	**e**	**f**	**g**		**cdmmmmmmef**	**100gttttttttttttttttttttttttttttttttttttttttttttttttttttt** Finite number with big first digit of significand (8 or 9).
combination field starting with '1111', bits abcd = 1111															
1	1	1	1	0											±Infinity
1	1	1	1	1	0										quiet NaN
1	1	1	1	1	1										signaling NaN (with payload in significand)

The leading bits of the significand field do *not* encode the most significant decimal digit; they are simply part of a larger pure-binary number. For example, a significand of 8 000 000 000 000 000 is encoded as binary 0111 0001101011 1111010100 1001100011 0100000000 0000000000$_2$ with the leading 4 bits encoding 7; the first significand which requires a 54th bit is 2^{53} = 9 007 199 254 740 992. The highest valid significant is 9 999 999 999 999 999 whose binary encoding is (100)0 1110000110 1111001001 1011111100 0000111111 1111111111$_2$ (with the 3 most significant bits (100) not stored but implicit as shown above; and the next bit is always zero in valid encodings).

In the above cases, the value represented is

$$(-1)^{\text{sign}} \times 10^{\text{exponent}-398} \times \text{significand}$$

If the four bits after the sign bit are "1111" then the value is an infinity or a NaN, as described above:

```
0 11110 xx...x    +infinity
1 11110 xx...x    -infinity
x 11111 0x...x    a quiet NaN
x 11111 1x...x    a signalling NaN
```

Densely packed decimal significand field

In this version, the significand is stored as a series of decimal digits. The leading digit is between 0 and 9 (3 or 4 binary bits) and the rest of the significand uses the densely packed decimal (DPD) encoding.

The leading 2 bits of the exponent and the leading digit (3 or 4 bits) of the significand are combined into the five bits that follow the sign bit. The eight bits after that are the exponent continuation field, providing the less-significant bits of the exponent. The last 50 bits are the significand continuation field, consisting of five 10-bit declets.[3] Each declet encodes three decimal digits[3] using the DPD encoding.

If the initial two bits following the sign bit are "00", "01", or "10", they represent the leading bits of the exponent, whereas the subsequent three bits "cde" are regarded as the leading decimal digit (ranging from 0 to 7):

If the first two bits after the sign bit are "11", then the second 2-bits are the leading bits of the exponent, and the next bit "e" is prefixed with implicit bits "100" to form the leading decimal digit (8 or 9):

The remaining two combinations (11 110 and 11 111) of the 5-bit field after the sign bit are used to represent ±infinity and NaNs, respectively.

DPD Encoding

Combination Field													Exponent	Significand / Description
g12	g11	g10	g9	g8	g7	g6	g5	g4	g3	g2	g1	g0		
combination field not starting with '11', bits ab = 00, 01 or 10														
a	**b**	**c**	**d**	**e**	**m**	**m**	**m**	**m**	**m**	**m**	**m**	**m**	**abmmmmmmmm**	(0)**cde tttttttttt tttttttttt tttttttttt tttttttttt tttttttttt** Finite number with small first digit of significand (0 … 7).
combination field starting with '11', but not 1111, bits ab = 11, bits cd = 00, 01 or 10														
1	1	**c**	**d**	**e**	**m**	**m**	**m**	**m**	**m**	**m**	**m**	**m**	**cdmmmmmmmm**	**100e tttttttttt tttttttttt tttttttttt tttttttttt tttttttttt** Finite number with big first digit of significand (8 or 9).
combination field starting with '1111', bits abcd = 1111														
1	1	1	1	0										±Infinity
1	1	1	1	1	0									quiet NaN
1	1	1	1	1	1									signaling NaN (with payload in significand)

The DPD/3BCD transcoding for the declets is given by the following table. b9...b0 are the bits of the DPD, and d2...d0 are the three BCD digits.

Densely packed decimal encoding rules[4]

DPD encoded value											Decimal digits					
Code space (1024 states)	b9	b8	b7	b6	b5	b4	b3	b2	b1	b0	d2	d1	d0	Values encoded	Description	Occurrences (1000 states)
50.0% (512 states)	**a**	**b**	**c**	**d**	**e**	**f**	0	**g**	**h**	**i**	0**abc**	0**def**	0**ghi**	(0–7) (0–7) (0–7)	**3** small digits	51.2% (512 states)
37.5% (384 states)	**a**	**b**	**c**	**d**	**e**	**f**	1	0	0	**i**	0**abc**	0**def**	100**i**	(0–7) (0–7) (8–9)	**2** small digits, **1** large digit	38.4% (384 states)
	a	**b**	**c**	**g**	**h**	**f**	1	0	1	**i**	0**abc**	100**f**	0**ghi**	(0–7) (8–9) (0–7)		
	g	**h**	**c**	**d**	**e**	**f**	1	1	0	**i**	100**c**	0**def**	0**ghi**	(8–9) (0–7) (0–7)		
9.375% (96 states)	**g**	**h**	**c**	0	0	**f**	1	1	1	**i**	100**c**	100**f**	0**ghi**	(8–9) (8–9) (0–7)	**1** small digit, **2** large digits	9.6% (96 states)
	d	**e**	**c**	0	1	**f**	1	1	1	**i**	100**c**	0**def**	100**i**	(8–9) (0–7) (8–9)		
	a	**b**	**c**	1	0	**f**	1	1	1	**i**	0**abc**	100**f**	100**i**	(0–7) (8–9) (8–9)		
3.125% (32 states, 8 used)	*x*	*x*	**c**	1	1	**f**	1	1	1	**i**	100**c**	100**f**	100**i**	(8–9) (8–9) (8–9)	**3** large digits, b9, b8: *don't care*	0.8% (8 states)

Representation of every number from 0 to 999

The 8 decimal values whose digits are all 8s or 9s have four codings each. The bits marked x in the table above are ignored on input, but will always be 0 in computed results. The 8 × 3 = 24 non-standard encodings fill in the gap between $10^3 = 1000$ and $2^{10} = 1024$.

In the above cases, with the *true significand* as the sequence of decimal digits decoded, the value represented is

$$(-1)^{\text{signbit}} \times 10^{\text{exponentbits}_2 - 398_{10}} \times \text{truesignificand}_{10}$$

Implementations

There are multiple libraries available for calculations with decimal data types. Some examples are listed below:

- Libdfp implements the ISO/IEC Technical Report "ISO/IEC TR 24732". The library implements math functions for environments built on gcc and glibc. It uses BID-encoded IEEE 754 decimal types, with the possibility to switch to Decimal64 floating-point format via recompilation of gcc. The library can be slow, and sometimes inaccurate, for conversions and complicated operations.
- Intel Decimal Floating-Point Math Library is a C library that implements the IEEE 754-2008 Decimal Floating-Point Arithmetic specification by addressing decimal types as `UINT`*xx* items named `decimal`*xx*. The library improves performance by calculating more intricate functions in binary where possible, but this introduces some binary rounding errors into decimal calculations.
- decNumber provides data types and functions to calculate values using Decimal Floating-Point with arbitrary precision. It features the option to set the IEEE 754-compatible parameters `decSingle`, `decDouble`, and `decQuad`. It efficiently performs conversions between decimal numbers and strings.
- Mpdecimal is an implementation of the *General Decimal Arithmetic Specification* proposed by Mike Cowlishaw, with the option to set IEEE type-compatible parameters. It provides arbitrary precision, using an 8-bit integer to encode runtime flags and the number's sign. The library represents numbers using seven or more double- or quad-words, storing the signed exponent, the digits, the length in words, the allocated words, a pointer to `data`, and `data` itself. `Data` is stored as an array of two or more words, which holds the coefficient in 32-bit (9-digit) or 64-bit (19-digit) items. The library can handle a wide range of inputs and produces accurate results. It is implemented in Python as the `decimal` module.

See also

- ISO/IEC 10967, Language Independent Arithmetic
- Primitive data type
- D notation (scientific notation)

References

1. IEEE Computer Society (2008-08-29). *IEEE Standard for Floating-Point Arithmetic*. IEEE. doi:10.1109/IEEESTD.2008.4610935 (https://doi.org/10.1109%2FIEEESTD.2008.4610935). ISBN 978-0-7381-5753-5. IEEE Std 754-2008.
2. ISO/IEC JTC 1/SC 25 (June 2011). *ISO/IEC/IEEE 60559:2011 – Information technology – Microprocessor Systems – Floating-Point arithmetic* (https://www.iso.org/standard/57469.html). ISO. pp. 1–58.
3. Muller, Jean-Michel; Brisebarre, Nicolas; de Dinechin, Florent; Jeannerod, Claude-Pierre; Lefèvre, Vincent; Melquiond, Guillaume; Revol, Nathalie; Stehlé, Damien; Torres, Serge (2010). *Handbook of Floating-Point Arithmetic* (https://cds.cern.ch/record/1315760) (1 ed.). Birkhäuser. doi:10.1007/978-0-8176-4705-6 (https://doi.org/10.1007%2F978-0-8176-4705-6). ISBN 978-0-8176-4704-9. LCCN 2009939668 (https://lccn.loc.gov/2009939668).
4. Cowlishaw, Michael Frederic (2007-02-13) [2000-10-03]. "A Summary of Densely Packed Decimal encoding" (http://speleotrove.com/decimal/DPDecimal.html). IBM. Archived (https://web.archive.org/web/20150924145411/http://speleotrove.com/decimal/DPDecimal.html) from the original on 2015-09-24. Retrieved 2016-02-07.

Retrieved from "https://en.wikipedia.org/w/index.php?title=Decimal64_floating-point_format&oldid=1339544306"